

From Sound Waves to Web Pages: A Journey Through the Network Stack

Yuxin Fan
2023533044
fanyx2023@shanghaitech.edu.cn

January 11, 2026

Abstract

This report summarizes the design, implementation, and challenges encountered during the CS120 Computer Networks course projects. Over the course of four consecutive projects, a functional network stack was constructed from the bottom up. Starting with physical signal processing over audio channels, the work progressed to link-layer framing and encoding, network-layer routing and tunneling, and finally, transport-layer reliability and application-layer protocols including DNS and HTTP. This hands-on experience provided deep insights into the layered network architecture, emphasizing the importance of abstraction, error handling, and the complex interaction between user-space code and the operating system kernel.

1 Introduction

The internet is often viewed as a black box that simply "works." However, constructing it from scratch reveals the intricate dance of protocols required to move a single byte of data. The development journey began not with packets, but with raw analog signals, and concluded with a high-level HTTP client capable of browsing real-world websites. This report details the four stages of this development, focusing on the specific architectural decisions made, the hurdles overcome—particularly regarding synchronization and OS integration—and the final integration of a TCP/IP stack running entirely in user space.

2 Project 1: The Physical Layer

2.1 Overview: Making Bits Audible

The foundational project aimed to establish a communication channel using a medium inherently unsuited for high-speed digital data: audible sound. The primary objective was to transmit binary data between two nodes using standard audio files (WAV format) as the physical medium, effectively treating the computer's sound card as a Network Interface Card (NIC). This project underscored the analog nature of physical transmission and the necessity of converting digital discrete signals into continuous analog waveforms.

2.2 Design and Implementation

The architecture was composed of two distinct components: a transmission module and a reception module.

Transmission Logic The transmitter was responsible for converting a binary input stream (representing text or files) into an audio signal. A modulation scheme, especially On-Off Keying (OOK) when wired and Frequency Shift Keying (FSK) when transmitted wirelessly, was implemented to map bits to specific audio properties. To facilitate receiver synchronization, a standard preamble was prepended to the payload. This preamble served as a recognizable flag that signaled the beginning of a data packet within the continuous stream of audio samples.

Reception Logic The receiver's role was to process the incoming raw audio stream—often polluted with silence or background noise—and reconstruct the original binary sequence. The core of this logic involved signal processing techniques. By analyzing the energy levels or performing frequency domain analysis (such as Fast Fourier Transform) on small windows of the audio samples, the receiver could decide whether a bit was a '0' or a '1'. A visualization tool was also developed to aid in debugging, allowing for the graphical inspection of the waveform to identify signal degradation.

2.3 Challenges and Solutions

The most significant hurdle encountered was **signal synchronization and noise handling**. Unlike ideal digital simulations, real-world audio channels are noisy, and the receiver does not inherently know when the transmission begins.

Challenge: Preamble Detection Determining the precise start index of the data was difficult because the audio file contained leading silence. Simple energy thresholding was unreliable due to potential noise spikes.

Solution: A correlation-based preamble detection algorithm was implemented. The receiver continuously scanned the incoming audio stream, calculating the cross-correlation between the received signal and the known reference preamble pattern. When the correlation coefficient exceeded a predefined threshold, the system "locked on" to that position as the start of the frame. This ensures that the subsequent decoding process was aligned correctly with the bit boundaries.

3 Project 2: The Data Link Layer

3.1 Overview: From Bits to Frames

With a raw physical channel established, the next challenge was ensuring reliability and framing. A raw stream of decoded bits is insufficient for networking because it lacks boundaries (where does a byte start?) and error control (is this bit correct?). Project 2 introduced the "Aethernet" Data Link Layer to solve these issues.

3.2 Design and Implementation

The implementation shifted from signal processing to protocol logic, focusing on transforming a raw bitstream into structured, verifiable frames.

Framing and Addressing To distinguish legitimate data from random noise and to support multiple nodes, a frame structure was defined. Each frame consisted of a header containing the Destination MAC address, Source MAC address, and payload length, followed by the actual data payload. This allowed the receiver to process only frames intended for it.

4B/5B Encoding A 4B/5B encoding scheme was implemented to address clock recovery issues. In the raw physical layer, a long sequence of zeros could result in a long period of silence, causing the receiver to lose track of bit boundaries (clock drift). By mapping every 4 bits of data to a 5-bit symbol, the encoding ensured that there were sufficient state transitions to maintain synchronization, regardless of the data content.

Reliability Protocol (ARQ) To guarantee data delivery, a Stop-and-Wait Automatic Repeat reQuest (ARQ) protocol was implemented. The sender transmits a single frame and starts a timer. It waits for an explicit Acknowledgment (ACK) frame from the receiver before sending the next frame. If the timer expires (timeout) without receiving an ACK, the sender automatically retransmits the frame.

3.3 Challenges and Solutions

Challenge: Data Corruption and State Desynchronization Physical layer noise often flipped individual bits, corrupting the frame. Additionally, packets (or ACKs) could be lost entirely, leading to potential duplicate processing.

Solution: To handle corruption, a 32-bit Cyclic Redundancy Check (CRC) was integrated into the frame trailer. Upon reception, the CRC is strictly verified; failing frames are silently discarded, relying on the sender's timeout to handle the loss. To handle duplicates (e.g., sender retransmits because the ACK was lost), a 1-bit sequence number was added to the header. The receiver tracks the expected sequence number and discards any frames that match the previously received sequence number, ensuring that the application layer receives a clean, non-duplicated data stream.

4 Project 3: The Network Layer

4.1 Overview: Building the Router

Moving up the stack, Project 3 focused on internetworking. The goal was to bridge the custom "Aethernet" (Layer 2) with the host operating system's standard networking stack (Layer 3), effectively creating a software router that could forward IP packets between a virtual network interface and the custom link layer.

4.2 Design and Implementation

The centerpiece of this project was the router module, which operated in user space but interacted closely with the OS kernel.

Virtual Network Interface (TUN/TAP) A TUN (Tunnel) interface was created to intercept IP traffic. This virtual device allowed the Python script to read raw IP packets sent by the OS kernel and write legitimate IP packets back to the kernel. To the OS, this script appeared as a hardware network driver.

Routing Logic and ARP The router acted as a gateway. When an IP packet arrived from the TUN interface, the router had to decide where to forward it. An Address Resolution Protocol (ARP) mechanism was implemented to map logical IP addresses (e.g., internal virtual IPs) to physical MAC addresses. The router maintained a dynamic ARP table, populating it by observing incoming traffic or querying destinations.

Network Address Translation (NAT) To allow the isolated virtual node to access the real internet (e.g., pinging 1.1.1.1), Network Address Translation (NAT) was implemented. Since the virtual IP addresses were not routable on the real network, the router masqueraded as the host machine.

4.3 Challenges and Solutions

The complexity increased significantly as the system was now required to parse and manipulate standard IPv4 headers.

Challenge: Routing Loops and TTL Initial implementations faced issues where packets would bounce indefinitely between nodes or the kernel.

Solution: Standard Time-To-Live (TTL) processing was implemented. For every packet traversing the router, the TTL field was decremented. If the TTL reached zero, the packet was dropped, preventing infinite routing loops.

Challenge: NAT Implementation details Implementing NAT required complex state tracking.

Solution: A NAT translation table was designed to track sessions based on (Protocol, Source IP, Source Port). When an internal packet (e.g., from a virtual client) was forwarded to the external Wi-Fi interface, its source IP was rewritten to the host's actual IP, and its source port was mapped to a new, unique port managed by the router. Conversely, incoming traffic from the internet was matched against this table to determine the correct internal destination, rewriting the destination IP and port before forwarding it back to the virtual network.

5 Project 4: Transport and Application Layers

5.1 Overview: Reliability and the Web

The final phase focused on implementing connection-oriented transport (TCP) and application-layer protocols (DNS, HTTP) entirely in user space. This transformed the simple packet-forwarding system into a fully functional web agent capable of interacting with real-world web servers.

5.2 Design and Implementation

DNS Spoofing and Resolution Originally relying on a static hosts file, the system was upgraded to support dynamic DNS resolution. The router was modified to inspect all UDP traffic on port 53. When a DNS Query was detected, the router intercepted it, extracted the domain name, and used the host's standard resolver to find the real IP address. It then synthesized a valid DNS Response packet containing the resolved IP and injected it back into the network, allowing the client to resolve real-world domains like `example.com`.

User-Space TCP Stack A TCP client was implemented from scratch using raw packet construction. This involved managing the TCP state machine: sending a SYN packet to initiate a connection, waiting for a SYN+ACK, and responding with an ACK to complete the 3-way handshake.

HTTP Client Built on top of the TCP connection, a lightweight HTTP client was developed. It constructed compliant HTTP/1.0 GET requests, including necessary headers such as `Host` and `User-Agent`, and parsed the received HTML responses.

5.3 Challenges and Solutions

This phase presented the most subtle and difficult debugging challenge of the entire course: **The Kernel RST Conflict**.

Challenge: The Reset War When the user-space Python script sent a raw TCP SYN packet to an external server, the server correctly replied with a SYN+ACK. However, this reply was received by both the Python script *and* the host operating system's kernel. Since the kernel had no record of a socket connection on that specific source port (because the packet was generated raw in user space), it interpreted the SYN+ACK as an error (a packet for a closed port). In compliance with the TCP protocol, the kernel immediately generated and sent a RST (Reset) packet to the server. This RST packet often reached the server before the Python script could send the final ACK, causing the server to tear down the connection immediately.

Solution: Firewall Intervention To resolve this, the host's firewall rules were manipulated. A filter was created to silently drop any outgoing RST packets destined for the specific target server. This effectively "blinded" the kernel's interference mechanisms without affecting the user-space script's ability to send or receive other packets. Additionally, careful attention was paid to TCP Checksum recalculation in the NAT layer. Since

creating a new TCP packet (or modifying one via NAT) invalidates the checksum, a failure to recompute it led to silent packet drops by remote servers, which initially manifested as mysterious timeouts. Fixing the checksum logic ensured end-to-end connectivity.

6 Conclusion

This series of projects served as a rigorous deconstruction of the network stack. By implementing encoding schemes, the physical cost of synchronization became apparent. By writing reliable ARQ protocols, the complexity of error-free transmission was demonstrated. By building a NAT router and a raw TCP client, the fragility of the connection "handshake" was revealed, highlighting the immense workload typically handled by the OS kernel. The journey from processing raw audio signals to receiving a "200 OK" from a remote web server was a profound demonstration of the power and elegance of layered network abstraction.